Parallel Programming, 2.4

Совцов Илья, 24126

Benchmarks for this solution were run on Intel Core i9-13900H with 20 threads. All implementations are measured with thread counts of 1, 2, 4, 20 and 40.

Easy

ThreadUnsafeCounter

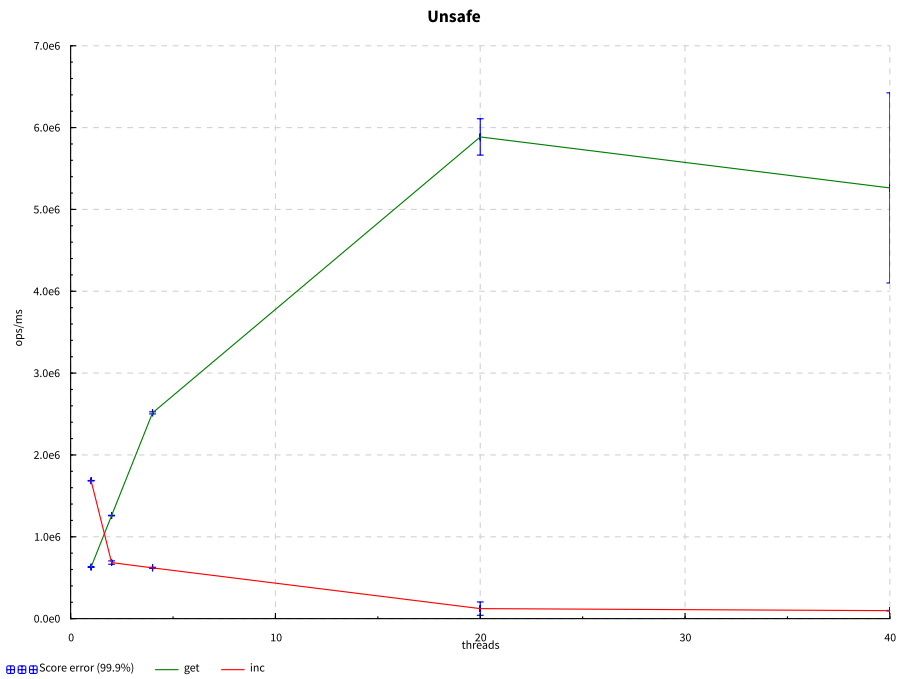

Figure 1: Correlation of thread count and `ThreadUnsafeCounter` throughput

Measurement results show that an increase in CPU count decreases throughput of `increment` operations and increases throughput of `get` operations. This scaling is pretty consistent until we reach the hardware thread limit, after that point the implementation does not scale.

Throughput stays about the same in case of over-subscription: results with 20 threads and 40 threads differ within the margin of error. At least this is not losing throughput.

LockedCounter

Throughput of `LockedCounter` with an unfair lock shows a general decline trend when CPU count increases. `get` and `increment` operations lose speed in the same way, because they are pretty cheap by themselves and both use a single `lock` + `unlock` pair. Oversubscription case throughput also differs from 1:1 thread mapping case within the margin of error. The only thing we see is a performance **decrease** when adding more threads.

But there's an interesting detail: two-thread case performs the worse. This, as far as my research goes, is due to an effect known as "thread ping-ponging". My guess is

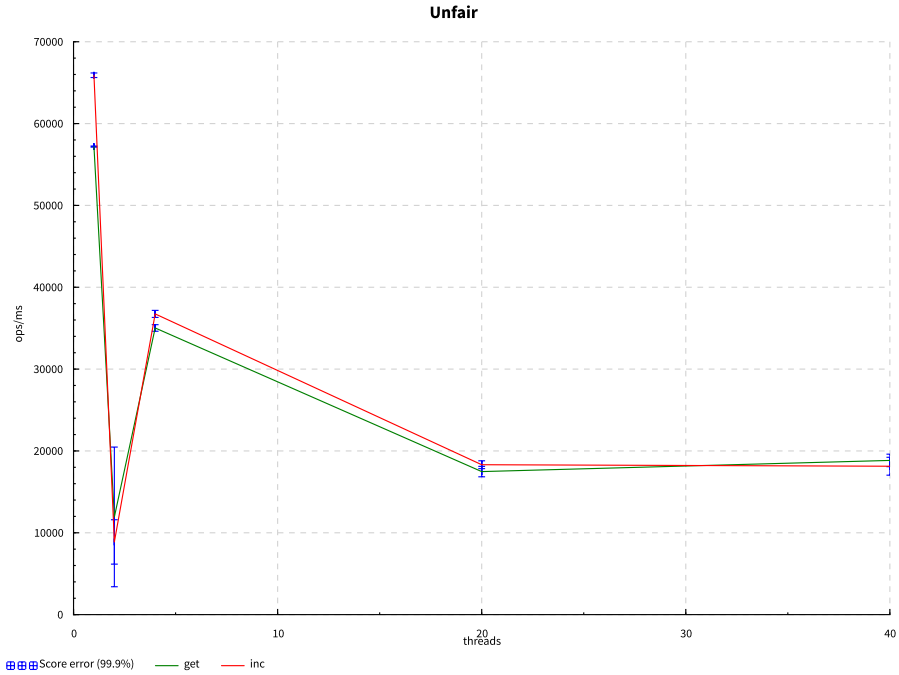

Figure 2: Correlation of thread count and `LockedCounter(fair=false)` throughput

that because the benchmark has only two threads and threads switch reapidly, because our critical section is quite small, OS frequently moves them between hardware threads, which leads to higher overhead.

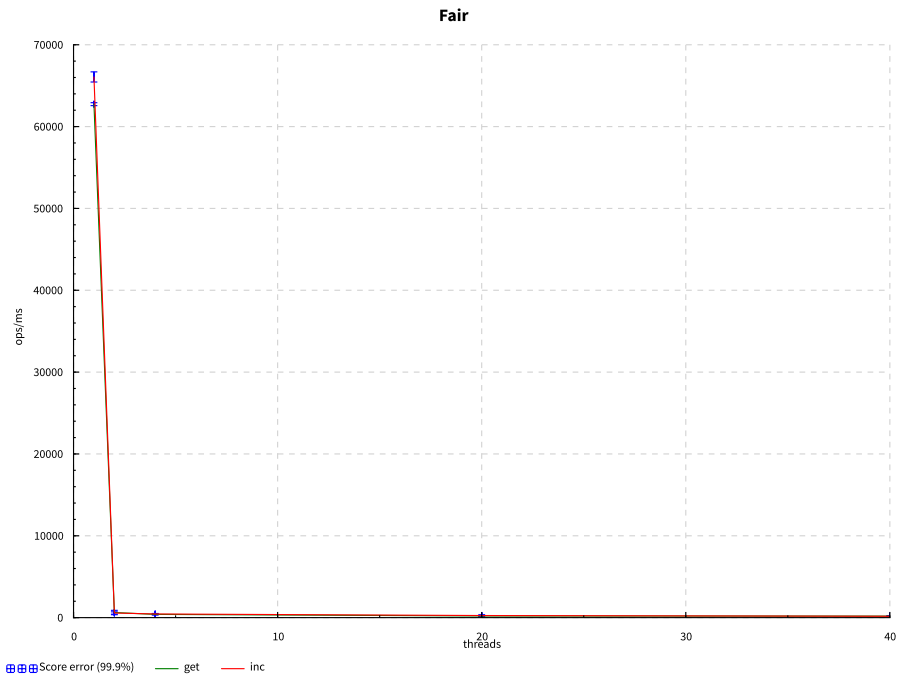

Figure 3: Correlation of thread count and `LockedCounter(fair=true)` throughput

`LockedCounter` with a fair lock shows a rapid throughput loss when thread count goes above one and stays consistently low. This cannot be called scaling, because throughput is really terrible. Over-subscription does not change much, the bar is low already.

Performance gap between fair and unfair implementations is present due to lower scheduling quantum utilization in the fair implementation. Because our critical section is very very small, fair implementation of lock leads to frequent context switches instead of letting a thread acquire the lock a few times in a row before switching after its quantum is over.

Also there is a substantial throughput gap between these implementations and the unsafe one. This is due to the additional overhead of acquiring the lock, which is not free (and not cheap either).

Medium

Correctness

There is no case when two threads access an item in `value` array simultaneously. Mutual exclusion guarantees that.

No deadlock can happen, simply because no lock is acquired **inside** any critical section.

Scalability

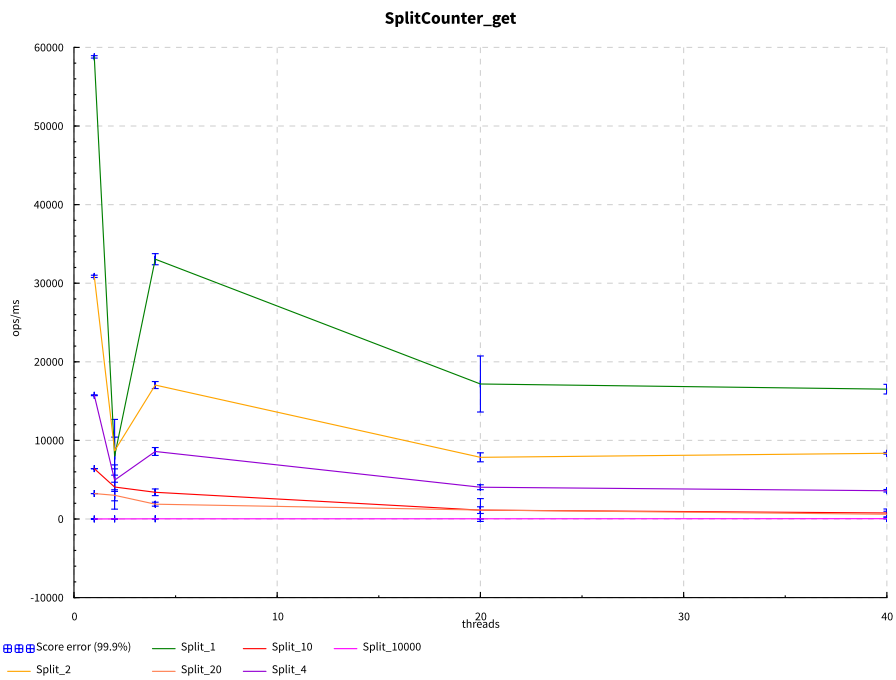

Figure 4: Correlation of thread count, granularity and `SplitCounter.get` throughput

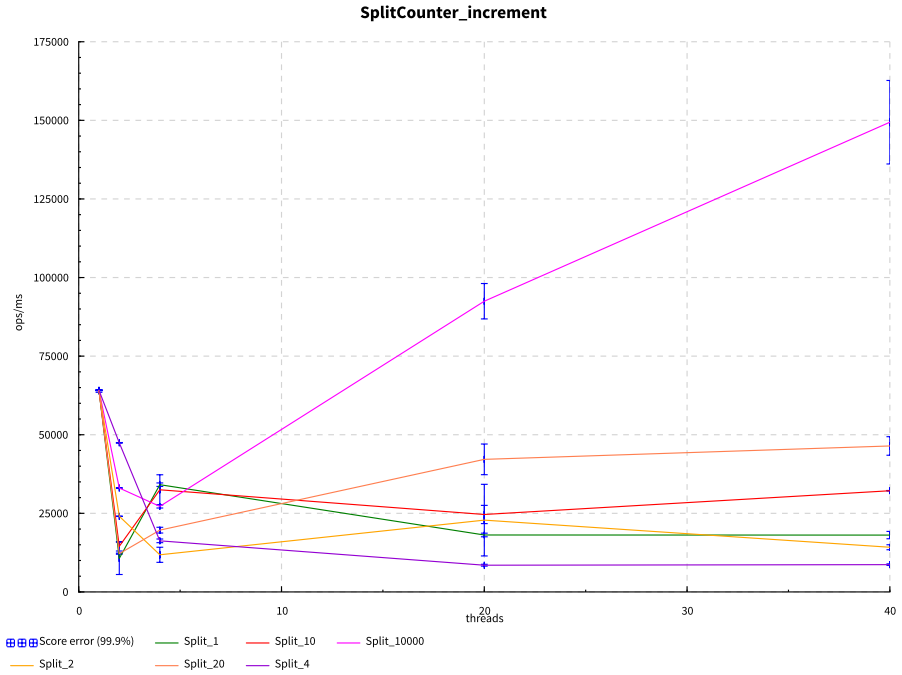

Figure 5: Correlation of thread count, granularity and `SplitCounter.increment` throughput

Whether we can cosider this implementation scalable depends on our use case: if there are significantly more `increments` than `gets`, then it scales pretty good. Even in the case of over-subscription `increment`'s throughput continues to increase. But if there are significantly more `gets`, then increasing granularity drastically affects performance. That is because the overhead of acquiring all locks gets higher and higher, and when `inrement` acquires one lock per call, `get` should acquire all of them, thus if several threads try to `get` at the same time, there will be more cases of them competing for a lock.

Aritifical contention problems can be forced by calling `get` from the threads whose `id % GRANULARITY` is the same. This is similar to collisions in hashmaps and can be alleviated using dynamic "rehashing": reassigning portions to threads that contend a lot, to distribute them across the locks more evenly.

Hard

BulkSplitCounter

No `increments` are lost, because update of any portion is done after acquiring the corresponding lock and before releasing it. Two calls to `idx` do not break anything, simply because a routine cannot jump between **application** threads, and context switches between hardware threads do not affect the ID of an application thread.

Neither `increment` nor `get` can deadlock. There are no blocking methods inside the critical section of `increment` and it eventually releases the lock. All `gets` start lock acquisition from the same (first) lock. This guarantees that only one `get` can happen

at a time. Because we already showed that `increment` eventually releases its lock, `get`, if it already progressed and tries to acquire a lock after the first one, will wait only for `increments`.

All `gets` can observe only non-negative values that are less than or equal to the sum of all arguments passed to `increments`.

To explain given examples lets number the operations in `increment` and `get`:

```
public void increment(long delta) {
    if (delta <= 0) { ... }
    for (long i = 0; i < delta; i++) {
        final var l = locks[idx()]; // 1.i
        l.lock();                   // 2.i
        try {
            portion[idx()]++;       // 3.i
        } finally {
            l.unlock();             // 4.i
        }
    }
}
public void get() {
    long result = 0;
    for (int i = 0; i < GRANULARITY; i++) locks[i].lock();      // 5.i
    for (int i = 0; i < GRANULARITY; i++) result += portion[i]; // 6.i
    for (int i = 0; i < GRANULARITY; i++) locks[i].unlock();    // 7.i
    return result;
}
```

Notation `K.i` means that the operation with number K is running on the ith iteration of the loop. Also for compactness less say that N equals `GRANULARITY`.

- `x.get() == 0` is possible. Let's take two threads, one of them is doing `get()`, the other is doing `increment(5)`. Execution trace

  $$5.1 \to \cdots \to 5.N \to 6.1 \to \cdots \to 6.N \to 7.1 \to \cdots \to 7.N \to 1.1 \to \ldots$$

  gives the desired output.

- `x.get() == 5` is possible. Let's take the same two threads as before. Execution trace

  $$1.1 \to 2.1 \to 3.1 \to 4.1 \to \cdots \to 1.N \to 2.N \to 3.N \to 4.N \to 5.1 \to \ldots$$

  gives the desired output.

- `x.get() == -1` is impossible. Getting negative numbers requires a data race, which is impossible because of mutual exclusion, and all counter modification in the code sample only increment it. Overflow is impossible because we assumed a single call to `increment(5)`.

- `x.get() == 3` is possible. Let's take the same two threads once again. Execution trace

$$1.1 \to 2.1 \to 3.1 \to 4.1 \to 1.2 \to \cdots \to 4.3 \to$$
$$\to 5.1 \to \ldots 5.N \to 6.1 \to \cdots \to 6.N \to 7.1 \to \cdots \to 7.N \to$$
$$\to 1.4 \to \ldots$$

  gives the desired output.

NegativeSplitCounter

Safety and progress guarantees are explained the same way as in previous part.

Where acquisition of one lock at a time is better for scalability depends on granularity. If granularity is small, locking everything leads to fewer context switches wile `get`ting, but as granularity grows, acquiring all the locks leads to a lot of waiting, because multiple `gets` cannot work in parallel.

This implementation of counter is not safe. Consider consider the case with an initial value of $x = y + z$ (y and z are "portions" of the value) and the following execution trace:

1. Thread A starts `get`. Acquires the first lock, loads the first "portion" of the value, y, releases the lock.

2. Context switch occures, thread B does `increment(-2)`, and it so happens that thread B is mapped to the first "portion" of the value. Now $x = y - 2 + z$.

3. Then another context switch occures, thread C does `increment(1)`, and it so happens that thread C is mapped to the second "portion" of the value. Now $x = y - 2 + z + 1 = y + z - 1$.

4. Context switch to thread A occures, A loads the second "portion" of the value, which is $z + 1$.

As a result, A got $y + z + 1$ from `get`, which never existed (all the values we saw were: $y + z$, $y + z - 2$ and $y + z - 1$).